

YOLO Object Recognition

Using OpenCV to call YOLOv11 for object recognition and detection, it can recognize the names of most objects.

1. Main Code

Code path:

```
/home/dofbot/Dofbot/6.AI_Visual/8.Object_recognition.ipynb
```

2. File Configuration

First, we need to import the YOLO11n.pt network model structure cfg file, network weights file, and the COCO dataset classification names txt file. (Here we directly use the official YOLOv4 dataset as an example)

3. Object Detection

Building Model Network Structure

First, use the **cv2.dnn.readNet()** function to construct the CSPDarknet53 network structure, passing in the model structure cfg file and network weights file. OpenCV provides support for image classification, detection, and segmentation for neural network modules, automatically implementing preprocessing and post-processing of input images. Here, the object detection module **cv2.dnn_DetectionModel()** is used to pass in the network model.

```
robot = Robot_Controller()
robot.move_init_pose()
fps = FPS()
yolo_model = YOLO("./yolo11n.pt", task='detect')
model = "General"
```

Object Detection Method

```
classids, scores, bboxes = self.model.detect(image, confThreshold,
numsThreshold)
```

Parameters:

frame: Input image

confThreshold: Confidence threshold for filtering selection boxes, minimum confidence for object detection

numsThreshold: Custom threshold in non-maximum suppression

Return values:

classids: Class index

confidences: Confidence, the probability that the detection box belongs to a certain class

boxes: Detection box information, upper-left coordinates (x, y), box width and height (w, h)

Setting Model Input Parameters

```
self.model.setInputParams(size=(320,320), scale=1/255)
```

size indicates the input image will be scaled to the specified size. Larger size means better detection effect but slower detection speed. **scale** indicates the scaling of pixel values.

Usage Testing

The following code content needs to be executed step by step according to actual needs, or you can click the run all button. For detailed code, please check the code path.

Import various libraries and model files

```
#!/usr/bin/env python
# coding: utf-8
import Arm_Lib
import cv2 as cv
import threading
from time import sleep
import ipywidgets as widgets
from IPython.display import display
from Object_recognition import Object_recognition_identify
```

Object recognition function

```
def camera():
    # Open camera
    capture = cv.VideoCapture(0)
    capture.set(cv.CAP_PROP_FRAME_WIDTH, 640)
    capture.set(cv.CAP_PROP_FRAME_HEIGHT, 480)
    # Loop while camera is open
    while capture.isOpened():
        try:
            _, img = capture.read()
            fps.update_fps()
            results = yolo_model(img, save=False, verbose=False) # Use YOLO11
            for object detection
                annotated_frame = results[0].plot(
                    labels = True, # Show labels
                    conf = False, # Show confidence
                    boxes = True, # Draw bounding boxes
                )
            if model == 'Exit':
                cv.destroyAllWindows()
                capture.release()
                break
            fps.show_fps(annotated_frame)
            imgbox.value = cv.imencode('.jpg', annotated_frame)[1].tobytes()
        except Exception as e:
            capture.release()
            print(e)
            break
```

Object name list:

```
1 person
2 bicycle
3 car
4 motorbike
5 aeroplane
6 bus
7 train
8 truck
9 boat
10 traffic light
11 fire hydrant
12 stop sign
13 parking meter
14 bench
15 bird
16 cat
17 dog
18 horse
19 sheep
20 cow
21 elephant
22 bear
23 zebra
24 giraffe
25 backpack
26 umbrella
27 handbag
28 tie
29 suitcase
30 frisbee
31 skis
32 snowboard
33 sports ball
34 kite
35 baseball bat
36 baseball glove
37 skateboard
38 surfboard
39 tennis racket
40 bottle
41 wine glass
42 cup
43 fork
```

Main thread

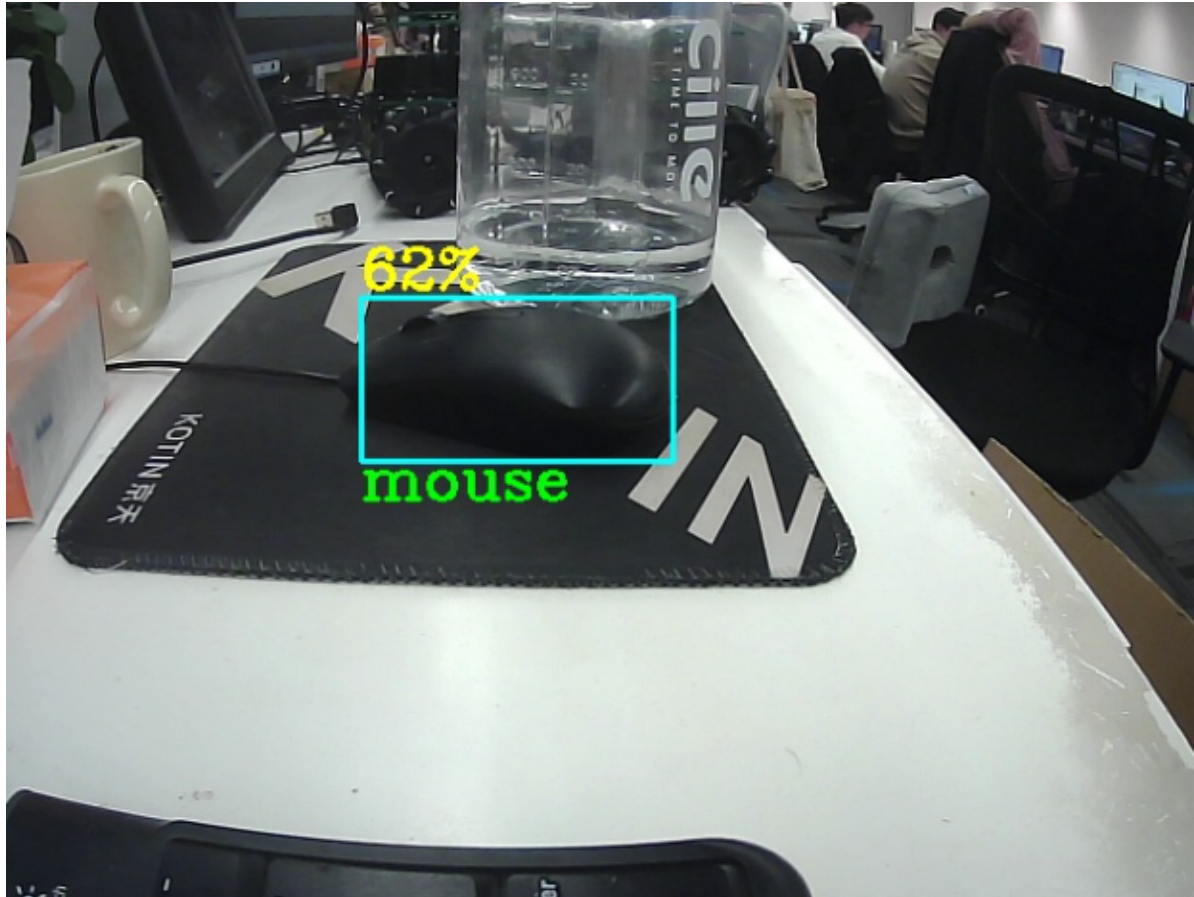
```
def camera():
    # Open camera
    capture = cv.VideoCapture(0)
    # Loop while camera is open
    while capture.isOpened():
        try:
            _, img = capture.read()
            img = cv.resize(img, (640, 480))
            img = ob_re.detect_image(img)
            if model == 'Exit':
```

```

cv.destroyAllWindows()
capture.release()
break
imgbox.value = cv.imencode('.jpg', img)[1].tobytes()
except KeyboardInterrupt:capture.release()

```

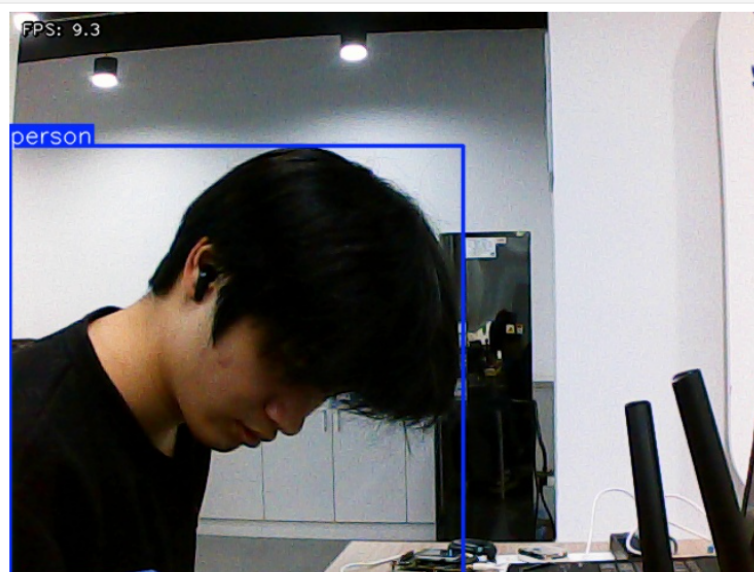
After running the code blocks, you can see the interface below with the corresponding objects recognized.



```

display(controls_box,output)
threading.Thread(target=camera, ).start()

```



Exit

[WARN:1] global ./modules/videoio/src/cap_gstreamer.cpp (1100) open OpenCV | GStreamer warning: Cannot query video position: status=0, val

